

- 人工智能实验报告 第5周
  - 任务三：
    - 一.实验题目：中国象棋
    - 二.实验内容
      - 1.算法原理
      - 2.关键代码展示(可选)
    - 三.实验结果及分析
      - 1.实验结果展示示例（可截图可表可文字，尽量可视化）
      - 2.评测指标展示及分析（机器学习实验必须有此项，其它可分析运行时间等）

# 人工智能实验报告 第5周

---

姓名:陈安康 学号:22320021

## 任务三：

---

### 一.实验题目：中国象棋

### 二.实验内容

#### 1.算法原理

在博弈树搜索上比较常用的算法就是MCTS和alpha-beta剪枝算法。由于题目代码内置的算法是采用alpha-beta剪枝算法，并且在给定代码框架下alpha-beta剪枝算法的实现思路都极其相似，并且性能差异十分微小，可开发的内容不多。可是在搜索范围较小并且确定性比较高的中国象棋场景下，更适合于确定性和完美信息下进行搜索的alpha-beta算法更加占有优势。因此我尝试集中这两个算法的优势进行结合，创造一个全新的算法，以下是我的思路：

已知alpha-beta剪枝算法在给定深度和初始场景（在这里是棋盘情况），先手方的情况下，得到的结果是唯一确定的。无论模拟多少次，你都会得到唯一确定的得分。这是alpha-beta算法确定性的显现。但是由于性能的原因，alpha-beta剪枝算法仅可以搜索特定的深度（在我的电脑上，最大搜索深度为5，如果为6，则会十分缓慢）并且每个节点搜索深度都一致，尽管有剪枝操作，但还是不可避免有资源浪费。\*\*在有限的运算资源

下，如果能够选定“更好”的场景进行进一步的搜索，而对其他节点进行忽略，那么就有可能得到更好的结果。**\*\*这和MCTS的想法基本一致，但是MCTS算法在模拟时往往是采用随机选择模拟的情况，不太符合中国象棋这种确定性和搜索范围小的场景，因此，把模拟从随机选择改为采用alpha-beta剪枝进行确定性搜索的思路就变得顺理成章了（这违背了MCTS的核心思想，即通过大量随机性模拟来获得结果。所以我只是借用了它的代码框架和其他想法，我只是借鉴而已）。**

借鉴MCTS的节点选择和扩展思路，我们可以这样设计算法：

- 1.给定初始节点（这里就是棋盘情况），先手方（这里全部为己方），alpha-beta剪枝算法深度 $depth_{max}$ ，模拟次数 $count$ 。
- 2.对于每个节点，若其未扩展，进行扩展，并且对该节点进行alpha-beta剪枝算法模拟。得到“得分”，传递“得分”到其祖先节点，结束这一次模拟。
- 3.否则，依据UCB选择其子节点中更好的节点，重复这个过程，直到遇到未扩展节点，执行操作2。
- 4.重复操作2和操作3直到达到规定模拟次数。

在上述算法中，“得分”是给定的得分函数以根节点的team所计算的数值，ucb计算公式如下：

$$UCB = \overline{score} + param * \sqrt{\frac{\ln(visited_{parent})}{visited}}$$

其中 $\overline{score}$ 是节点得分的平均值， $param$ 为设置的参数， $visited_{parent}$ 是其父节点的访问次数， $visited$ 是当前节点的访问次数。上述公式可以很好的平衡广度与深度搜索的比重。通过 $param$ 和 $visited$ 的设置（因为除数不能为0，所以期望中的无穷大是达不到的，需要给 $visited$ 设置一个很小的初始值，这给了我们控制后面一项初始数量级的机会），可以调整使算法倾向于遍及更多可能还是专注于当前的局部深度搜索。

还有一点值得注意的是，alpha-beta剪枝算法会出现自己先手还是自己后手的情况，这种情况下统一计算的是根节点的team的得分，然后按照当前模拟的节点的team来选择返回a还是b。

## 2.关键代码展示(可选)

仿照蒙特卡洛树搜索过程代码如下

```
def MCTS(self, chessboard, count):  
    hhhmax = 0
```

```

root = MY_AI2(self.team, None)
for _ in range(count):
    hhh = 1
    node = root
    while True:
        if len(node.children) == 0:
            node.expand(chessboard)
            break
        else:
            tmp_node = node.select_child()
            hhh += 1
            # if tmp_node == None:
            #     node.backpropagate(val, chessboard)
            #     best_child = max(root.children, key = lambda
c:c.val/c.visited)
            #     return best_child
            node = tmp_node
            chess = chessboard.chessboard_map[node.old_pos[0]]
[node.old_pos[1]]

            chess.update_position(node.op[0], node.op[1])
            chessboard.chessboard_map[node.op[0]][node.op[1]] = chess
            chessboard.chessboard_map[node.old_pos[0]][node.old_pos[1]] =
None

            #下面进行模拟，这个待定，用alpha-beta剪枝进行模拟
            val = node.simulate(chessboard, -1 * (0x3f3f3f3f),
(0x3f3f3f3f), 1, node.team, node.max_depth, root.team)
            hhh += self.max_depth
            #如果是对手操作，val是评估函数计算得到的值，这个值肯定是越大越好，但对我们来
说，这个值应给越小越好，所以这里应该有一个判断
            #if node.team != root.team:
            #max_val是一次得分的最大值
            #    val = -val
            node.backpropagate(val, chessboard)
            hhhmax = max(hhhmax, hhh)
            best_child = max(root.children, key = lambda c:c.val/c.visited)
            #best_child = root.select_child()
            print(hhhmax)
            return best_child

```

其中回溯（backpropagate）代码如下：

```

#这个函数还起着恢复现场的作用
def backpropagate(self, val, chessboard):
    self.visited += 1
    self.val += val
    if self.op != None and self.old_pos != None:
        chess = chessboard.chessboard_map[self.op[0]][self.op[1]]
        chess.update_position(self.old_pos[0], self.old_pos[1])
        chessboard.chessboard_map[self.old_pos[0]][self.old_pos[1]] = chess
        chessboard.chessboard_map[self.op[0]][self.op[1]] = self.chess_eat
    if self.parent:
        self.parent.backpropagate(val, chessboard)

```

# 三.实验结果及分析

## 1.实验结果展示示例（可截图可表可文字，尽量可视化）

算法的效率基本上取决于 $count$ 和 $depth_{max}$ 其中 $depth_{max}$ 对算法能力影响很大，但同时也会显著增加运行时间。在和代码给定的alpha-beta剪枝算法（下面称内置ai）进行对打时，在 $visited$ 初始给定为0.000001， $param$ 为1.4， $depth_{max}$ 给定为4， $ount$ 给定为70时对于内置ai设置最大深度为4时，若我先手（红色），则下面是结果图，下面是结果图：



若我为后手（黑色），则看来是我的赢面大，看棋面是我赢了，下面是结果图（这个获胜显示的位置有问题）：



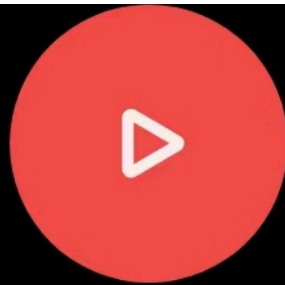
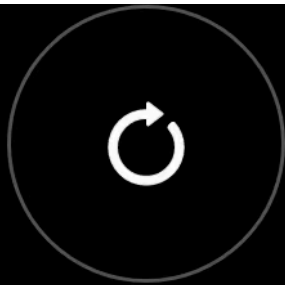
下面是每一步运行时间的截图，在一开始时，作出一步决策大概要7-13秒的时间，当棋子慢慢变少，作出决策时间大概为1-2秒：

# 一秒表



04:49.60

| 07 | +00:12.70 | 01:20.00 |
|----|-----------|----------|
| 06 | +00:13.18 | 01:07.52 |
| 05 | +00:10.69 | 00:54.34 |
| 04 | +00:08.40 | 00:43.65 |
| 03 | +00:14.83 | 00:35.25 |
| 02 | +00:13.17 | 00:20.42 |
| 01 | +00:07.25 | 00:07.25 |



闹钟



世界时钟



秒表



计时器

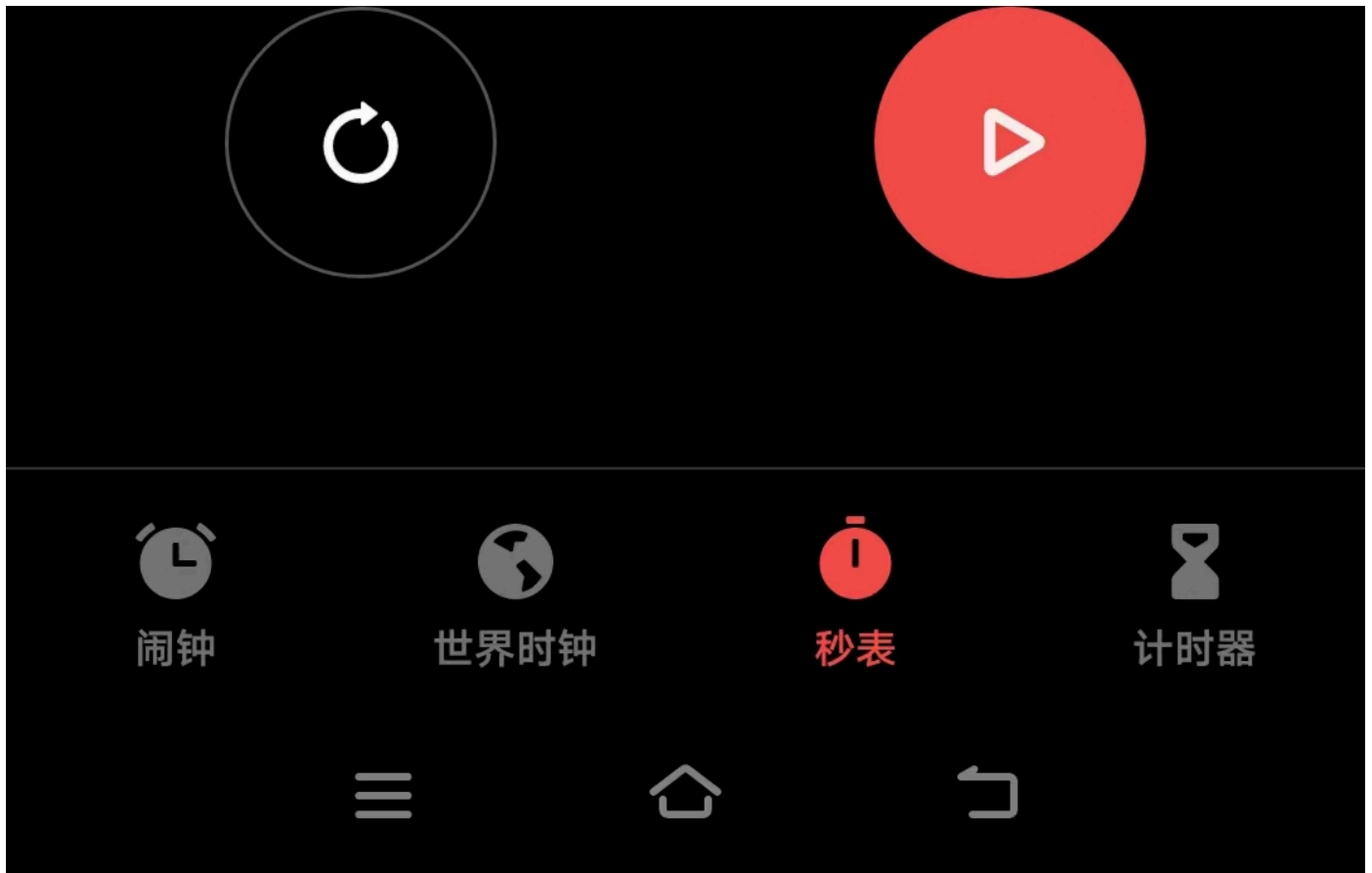


# 一秒表

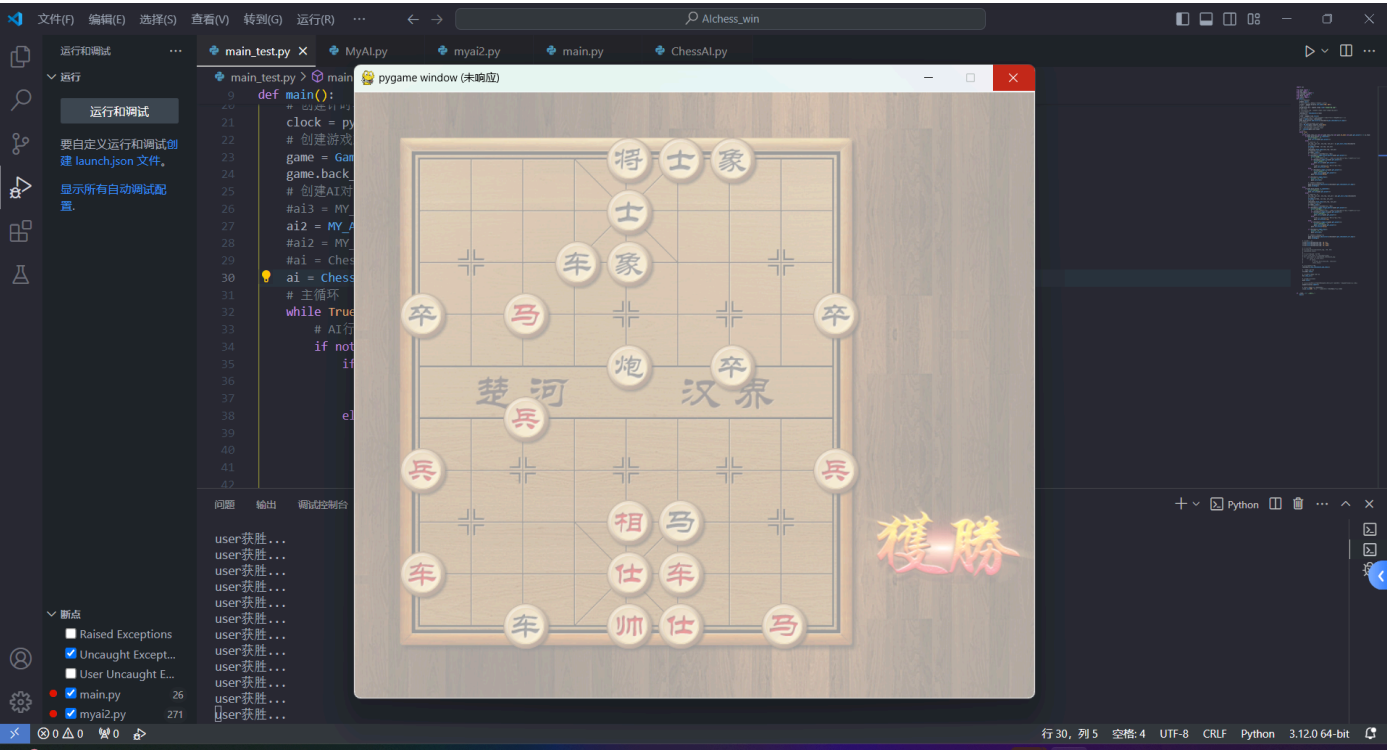


04:49.60

|    |           |          |
|----|-----------|----------|
| 49 | +00:01.43 | 04:41.26 |
| 48 | +00:01.52 | 04:39.83 |
| 47 | +00:01.61 | 04:38.31 |
| 46 | +00:01.97 | 04:36.70 |
| 45 | +00:02.09 | 04:34.73 |
| 44 | +00:02.23 | 04:32.64 |



对于内置ai搜索深度为5层时，在我先手（红色）的情况下，最后会因为重复走子或者被将死判负，而在我后手情况下（黑色）时，当修改count为80次时，我会稳定获胜，下面是截图（从下图来看，明显是黑色方（我）赢了，但获胜打在了红色方（内置ai），显然获胜的位置打印有问题，不能作为参考）：



下面是我作为红色方先手的时候的结果图，如果不重复走子继续下去，应该会和解：



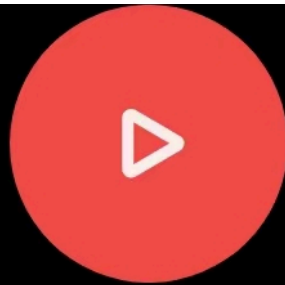
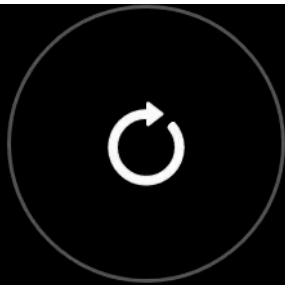
运行时间上，我的ai运行做出决策大概需要12-14秒的时间，后期大概5秒左右的时间作出决策，而五层ai需要1-4秒，虽然时间上不占优，但从算力结果来说，我的ai算力更加好一些，下面是运行时间的展示图，红黑方决策时间交叉展示：

# 一秒表



07:15.25

|    |           |          |
|----|-----------|----------|
| 12 | +00:04.02 | 01:58.65 |
| 11 | +00:13.37 | 01:54.63 |
| 10 | +00:01.48 | 01:41.26 |
| 09 | +00:14.22 | 01:39.78 |
| 08 | +00:06.03 | 01:25.56 |
| 07 | +00:14.74 | 01:19.53 |



闹钟



世界时钟



秒表



计时器

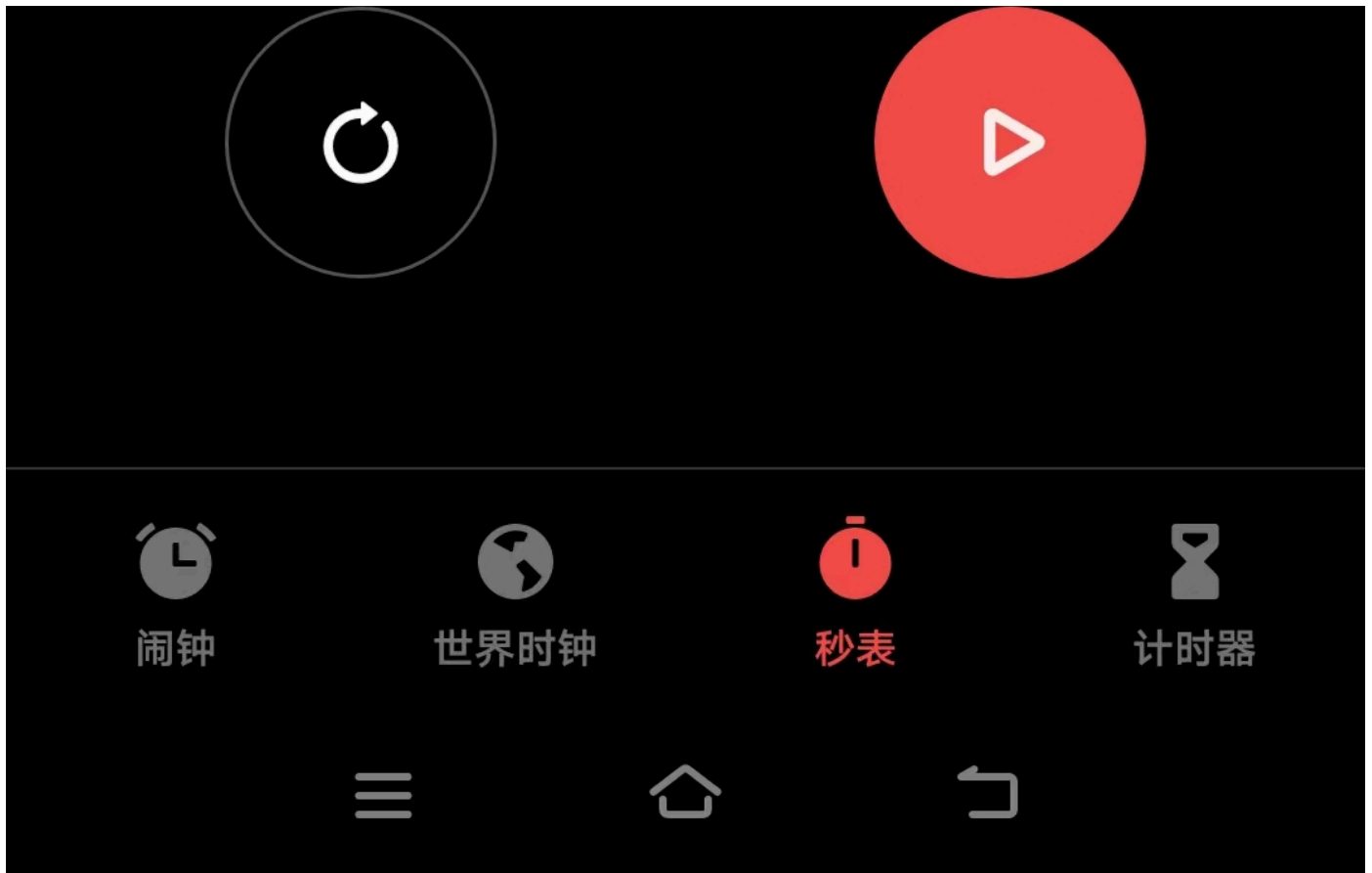


# 一秒表



07:15.25

|    |           |          |
|----|-----------|----------|
| 62 | +00:01.53 | 06:11.02 |
| 61 | +00:04.32 | 06:09.49 |
| 60 | +00:01.25 | 06:05.17 |
| 59 | +00:05.73 | 06:03.92 |
| 58 | +00:00.49 | 05:58.19 |
| 57 | +00:05.73 | 05:57.70 |



在搜索深度上，平均每次最大搜索深度可以到达8-10层，到后期棋子变少时可以达到18-25层甚至90多层，以下终端中输出的数字便是每次决策搜索的最大深度：

```
10
旧位置： 8 3 新位置： 9 3 9步双方没有互吃棋子
旧位置： 4 2 新位置： 5 2 10步双方没有互吃棋子
10
旧位置： 7 3 新位置： 1 3 11步双方没有互吃棋子
旧位置： 8 5 新位置： 9 5 12步双方没有互吃棋子
ai将军....
10
旧位置： 9 3 新位置： 8 3 13步双方没有互吃棋子
旧位置： 5 2 新位置： 6 2 0步双方没有互吃棋子
8
旧位置： 3 6 新位置： 2 8 1步双方没有互吃棋子
```

|     | 问题       | 输出  | 调试控制台 | 终端  | 端口         |
|-----|----------|-----|-------|-----|------------|
|     | 旧位置:     | 9 3 | 新位置:  | 8 4 | 1步双方没有互吃棋子 |
|     | 旧位置:     | 7 3 | 新位置:  | 7 2 | 0步双方没有互吃棋子 |
|     | 10       |     |       |     |            |
|     | 旧位置:     | 5 6 | 新位置:  | 5 2 | 1步双方没有互吃棋子 |
|     | 旧位置:     | 2 1 | 新位置:  | 9 1 | 2步双方没有互吃棋子 |
|     | 12       |     |       |     |            |
|     | 旧位置:     | 9 4 | 新位置:  | 9 3 | 3步双方没有互吃棋子 |
|     | 旧位置:     | 9 1 | 新位置:  | 9 2 | 0步双方没有互吃棋子 |
|     | ai将军.... |     |       |     |            |
|     | 95       |     |       |     |            |
| 26  | 旧位置:     | 9 3 | 新位置:  | 8 3 | 1步双方没有互吃棋子 |
| 271 | 旧位置:     | 9 2 | 新位置:  | 8 2 | 2步双方没有互吃棋子 |

## 2.评测指标展示及分析（机器学习实验必须有此项，其它可分析运行时间等）

虽然看似我的运行时间不占优势，但如果放长远一点，即内置ai如果搜索深度设置为6层时，它作出一步所需要的时间大概是1分钟到一分半，偶尔会有2分钟作出一次决策的情况。但我的ai在调整参数达到与之相同的运算能力的情况下（走成相互平局的情况），作出一次决策的运行时间平均是30-40秒，并且我的ai作出一次决策的时间花费十分稳定，而内置ai决策时间的波动比较大，我的ai的另一个优势就是算力调节更加精细，通过对参数的调节，可以精准获得各个算力情况下效果，并且决策时间的增长基本成线性(对同等搜索深度仅修改模拟次数而言，如果修改最大搜索深度，则也是成指数级)，而内置ai如果想要加深一层，运算时间的增长是指数级的。

基本上，若想达到alpha-beta剪枝算法5-6层之间的算力( $depth_{max} = 4, count = 80$ )大概我的每次决策时间是10-13秒。

若达到alpha-beta剪枝算法4-5层之间的算力( $depth_{max} = 4, count = 70$ )大概我的每次决策时间是8-10秒。

代码位于myai2.py中,实现两个ai对战的代码位于main\_test.py中。